

Fiche explicative des tâches — Sprint 1

EduTutor IA

Objectif du Sprint 1 : mettre en place les bases fonctionnelles du MVP autour de deux premières User Stories :

- **US-01 : Inscription utilisateur**
 - **US-02 : Upload / saisie d'un cours**
-

US-01 — Inscription utilisateur

T-01.1 — Modèle User Django

Assigné : Mazen

Estimation : 1h

Objectif

Préparer le modèle utilisateur côté Django pour permettre à un utilisateur de créer un compte.

Ce qu'il faut faire

Créer ou adapter le modèle utilisateur avec les champs nécessaires à l'inscription : email, mot de passe, date de création, et éventuellement rôle utilisateur si besoin.

Livrable attendu

Un modèle utilisateur propre, utilisable par Django Auth ou compatible avec l'authentification prévue.

Critère de validation

Le modèle doit permettre de créer un utilisateur sans erreur côté backend.

T-01.2 — Endpoint API signup

Assigné : Melvyn

Estimation : 2h

Objectif

Créer endpoint backend permettant à un utilisateur de s'inscrire depuis l'interface.

Ce qu'il faut faire

Créer un endpoint de type :

POST /api/signup

L'endpoint doit recevoir les informations d'inscription, vérifier les données, créer l'utilisateur et retourner une réponse claire.

Contraintes

Le mot de passe doit contenir au minimum 8 caractères.

L'email doit être valide.

Il ne faut pas créer deux comptes avec le même email.

Livrable attendu

Une route API fonctionnelle pour l'inscription.

Critère de validation

Quand on envoie un email et un mot de passe valides, un compte utilisateur est créé.

T-01.3 — Page React /signup

Assigné : Yoni

Estimation : 3h

Objectif

Créer la page d'inscription côté frontend.

Ce qu'il faut faire

Créer une page /signup avec un formulaire contenant au minimum :

- email
- mot de passe
- confirmation du mot de passe

- bouton d'inscription

La page doit appeler l'API d'inscription et afficher un message en cas d'erreur ou de succès.

Livrable attendu

Une page d'inscription utilisable par un utilisateur.

Critère de validation

Un utilisateur peut remplir le formulaire et envoyer ses informations au backend.

T-01.4 — Tests inscription

Assigné : Melvyn

Estimation : 1h

Objectif

Vérifier que l'inscription fonctionne correctement.

Ce qu'il faut faire

Tester les cas principaux :

- inscription avec données valides
- email invalide
- mot de passe trop court
- email déjà utilisé

Livrable attendu

Des tests ou une vérification documentée prouvant que l'inscription fonctionne.

Critère de validation

Les cas d'erreur sont bien gérés et l'inscription valide fonctionne.

T-01.5 — Critères d'acceptation inscription

Assigné : Mazen

Estimation : 1h

Objectif

Décrire clairement quand l'US-01 peut être considérée comme terminée.

Ce qu'il faut faire

Rédiger les critères d'acceptation de l'inscription, par exemple :

- l'utilisateur peut créer un compte avec un email valide
- le mot de passe doit faire au moins 8 caractères
- un message d'erreur apparaît si l'email existe déjà
- l'utilisateur est redirigé ou reçoit une confirmation après inscription

Livrable attendu

Une liste claire de critères d'acceptation pour US-01.

Critère de validation

L'équipe peut vérifier facilement si US-01 est terminée ou non.

US-02 — Upload / saisie d'un cours

T-02.1 — Modèle Course Django

Assigné : Mazen

Estimation : 1h

Objectif

Créer le modèle représentant un cours envoyé par un utilisateur.

Ce qu'il faut faire

Créer un modèle Course avec au minimum :

- utilisateur lié au cours
- titre
- contenu texte
- source du cours, par exemple PDF ou texte saisi
- date de création

Livrable attendu

Un modèle Course relié à l'utilisateur.

Critère de validation

Un cours peut être enregistré en base de données et associé au bon utilisateur.

T-02.2 — Endpoint upload PDF

Assigné : Anass

Estimation : 3h

Objectif

Permettre à l'utilisateur d'envoyer un fichier PDF comme cours.

Ce qu'il faut faire

Créer un endpoint de type :

POST /api/courses

L'endpoint doit accepter un PDF de 5 Mo maximum, extraire le texte du PDF et enregistrer le contenu dans le modèle Course.

Contraintes

Le fichier doit être un PDF.

Le fichier ne doit pas dépasser 5 Mo.

Si le PDF est invalide, une erreur claire doit être retournée.

Livrable attendu

Un endpoint backend permettant d'uploader un PDF et d'enregistrer son contenu.

Critère de validation

Un PDF valide est accepté, son contenu est extrait, puis sauvegardé.

T-02.3 — Endpoint texte $\geq$ 200 caractères

Assigné : Rizlène

Estimation : 1h

Objectif

Permettre à l'utilisateur de saisir directement son cours en texte sans PDF.

Ce qu'il faut faire

Ajouter ou vérifier la possibilité d'envoyer un texte directement via l'API.
Le texte doit contenir au minimum 200 caractères.

Contraintes

Si le texte fait moins de 200 caractères, une erreur doit être affichée.
Si le texte est valide, il est enregistré comme cours.

Livrable attendu

Une validation claire de la saisie texte.

Critère de validation

Un texte de 200 caractères ou plus est accepté, un texte trop court est refusé.

T-02.4 — Page React /upload

Assigné : Tahina

Estimation : 3h

Objectif

Créer la page permettant à l'utilisateur d'ajouter son cours.

Ce qu'il faut faire

Créer une page /upload avec deux possibilités :

- déposer un PDF
- saisir directement un texte dans une zone de texte

La page doit permettre de choisir la méthode d'import et envoyer les données au backend.

Livrable attendu

Une interface simple pour ajouter un cours.

Critère de validation

L'utilisateur peut soit déposer un PDF, soit saisir un texte, puis envoyer le cours.

T-02.5 — Tests upload + README

Assigné : Rizlène

Estimation : 1h

Objectif

Vérifier que la fonctionnalité d'upload est compréhensible et testable.

Ce qu'il faut faire

Tester les cas principaux :

- PDF valide
- PDF trop lourd
- mauvais format de fichier
- texte valide
- texte trop court

Ajouter une petite section dans le README pour expliquer comment tester l'upload.

Livrable attendu

Une vérification des cas principaux + documentation rapide.

Critère de validation

Un membre de l'équipe peut comprendre comment tester l'upload sans poser de question.

T-02.6 — Cohérence Story Map / Product Backlog / Release Planning

Assigné : Rizlène

Estimation : 1h

Objectif

Vérifier que les documents Scrum restent cohérents entre eux.

Ce qu'il faut faire

Comparer les éléments suivants :

- Story Map
- Product Backlog
- Sprint Backlog
- Release Planning

Vérifier que les US-01 et US-02 sont nommées de la même façon partout et qu'elles correspondent bien aux fonctionnalités du MVP.

Livrable attendu

Une vérification finale de cohérence entre les documents.

Critère de validation

Il ne doit pas y avoir de contradiction entre les fichiers : mêmes IDs, mêmes intitulés, même logique de priorité.
